۸-۶ اجتناب از بن‌بست

همان‌گونه که در بخش ۸-۵ بحث شد، الگوریتم‌های پیشگیری از بن‌بست با محدودکردن چگونگی انجام درخواست‌ها مانع وقوع بن‌بست‌ها می‌شوند. محدودیت‌ها تضمین می‌نمایند که حداقل یکی از شرایط لازم برای وقوع بن‌بست‌ها برقرار نمی‌شود. با این همه، بهره‌وری پایین دستگاه‌ها و کاهش بازده سیستم از عواقب پیشگیری بن‌بست‌ها محسوب می‌شوند.

روش دیگری که از بن‌بست‌ها اجتناب می‌ورزد مستلزم اطلاعات بیشتر در رابطه با چگونگی درخواست منابع می‌باشد. برای مثال، در سیستمی با منابع R1 و R2، سیستم نیاز دارد که بداند نخ P، ابتدا R1 و سپس R2 را درخواست خواهد نمود پیش از آنکه هر دو منبع را رها کند، در حالی‌که نخ Q، نخست R2 و سپس R1 را درخواست خواهد داد. سیستم با دانشی از دنباله‌ی کامل درخواست‌ها و رهاکردن‌های هر یک از نخ‌ها می‌تواند در مورد اعطای منابع و یا انتظار نخ با هدف اجتناب از بن‌بست احتمالی آتی تصمیم‌گیری نماید. سیستم برای اتخاذ چنین تصمیمی لازم است که برای هر درخواستی که صورت می‌گیرد منابع در دسترس فعلی، منابع در اختیار هر یک از نخ‌ها و درخواست‌ها و رهاکردن‌های آتی منابع هر یک از نخ‌ها را مد نظر قرار بدهد.

الگوریتم‌های مختلفی که از این رویکرد بهره می‌برند در حجم و نوع اطلاعاتی که لازم دارند با همدیگر فرق دارند. ساده‌ترین و کارآمدترین مدل لازم دارد که هر یک از نخ‌ها *تعداد حداکثر* هر یک از انواع منابع مورد نیاز خود را اعلام نمایند. با داشتن پیشاپیش این اطلاعات می‌توان الگوریتمی برپا کرد که اطمینان می‌دهد سیستم هرگز به حالت بن‌بست وارد نخواهد شد. الگوریتم اجتناب از بن‌بست، حالت تخصیص منابع را به صورت پویا بررسی می‌نماید تا این اطمینان را بدهد که هرگز شرط انتظار چرخشی به وجود نخواهد آمد. *حالت* تخصیص منابع توسط تعداد در دسترس، تخصیص‌یافته و حداکثر منابع مورد نیاز نخ‌ها تعریف می‌شود. در بخش‌های بعدی، دو الگوریتم اجتناب از بن‌بست را مطرح خواهیم کرد.

ابزار LOCKDEP لینوکس

اگرچه تضمین این که منابع به‌ترتیب درستی کسب می‌شوند مسئولیت برنامه‌نویسان هسته و کاربردی است، نرم‌افزار خاصی می‌تواند برای تصدیق این که قفل‌ها به‌ترتیب درست کسب می‌شوند استفاده شود. لینوکس برای تشخیص بن‌بست‌های احتمالی lockdep را فراهم می‌کند، ابزاری با کارکرد غنی که می‌تواند برای تصدیق ترتیب قفل‌گذاری در هسته استفاده شود. lockdep طراحی شده است تا در یک هسته‌ی در حال اجرا فعال شده و الگوهای استفاده از کسب و رهایی قفل‌ها را در مقابل یک مجموعه قوانین کسب و رهاسازی قفل‌ها کنترل کند. در ادامه دو مثال ارائه می‌شود، اما توجه داشته باشید که lockdep کارکرد بیشتری از آنچه که در اینجا بیان می‌شود فراهم می‌کند:

- ترتیبی که با آن قفل‌ها کسب می‌شوند به‌صورت پویا توسط سیستم نگه‌داری می‌شود. اگر lockdep قفل‌هایی را تشخیص بدهد که خارج از ترتیب کسب می‌شوند، یک شرط بن‌بست احتمالی را گزارش می‌دهد.
- در لینوکس، قفل‌های چرخشی می‌توانند در راه‌انداز وقفه‌ها استفاده شوند. یک منبع احتمالی بن‌بست زمانی است که هسته یک قفل چرخشی کسب می‌کند که در یک راه‌انداز وقفه نیز

استفاده می‌شود. اگر در حالی که قفل نگه داشته شده است، وقفه اتفاق بیفتد، راه‌انداز وقفه کد هسته‌ای را که در حال حاضر، قفل را نگه داشته است پس‌گرفته و سپس در حالی که تلاش می‌کند تا قفل را کسب کند می‌چرخد، نتیجه‌ی این کار بن‌بست است. راهبرد کلی برای اجتناب از این وضعیت، غیرفعال‌کردن وقفه‌ها در پردازنده‌ی فعلی است قبل از این که یک قفل چرخشی کسب شود که در یک راه‌انداز وقفه نیز استفاده می‌شود. اگر lockdep تشخیص دهد که وقفه‌ها فعال هستند، در حالی که کد هسته یک قفلی را کسب می‌کند که در یک راه‌انداز وقفه نیز استفاده می‌شود، یک سناریوی بن‌بست احتمالی را گزارش خواهد کرد.

lockdep توسعه داده شد تا به‌عنوان ابزاری در توسعه یا اصلاح کد هسته استفاده شود و نه در سیستم‌های تولید استفاده شود، چرا که می‌تواند سیستم را به‌صورت چشم‌گیری کند سازد. هدف آن این است که تست کند آیا نرم‌افزاری همچون یک درایور دستگاه جدید یا ماژول هسته یک منبع بن‌بست احتمالی هستند یا خیر. طراحان lockdep گزارش کرده‌اند که ظرف چند سال بعد از توسعه‌ی آن در سال ۲۰۰۶، تعداد بن‌بست‌های سیستم، ده مرتبه کاهش پیدا کرده است. هرچند که lockdep در ابتدا فقط برای استفاده در هسته طراحی شد، با اصلاحیه‌های اخیر این ابزار، اکنون می‌توان برای تشخیص بن‌بست‌ها در برنامه‌های کاربر استفاده‌کننده از قفل‌های انحصار متقابل نخ‌های پوزیکس استفاده کرد. جزئیات بیشتر مربوط به ابزار lockdep را می‌توانید در https://www.kernel.org/doc/Documnetation/locking/lockdep-design.txt بیابید.

۸-۶-۱ حالت امن

حالتی *امن* است که سیستم بتواند منابع را به ترتیبی به هر یک از نخ‌ها (تا حداکثر مورد نیاز آنها) تخصیص بدهد و همچنان از بن‌بست اجتناب بورزد. به بیان دقیق‌تر، سیستم در یک حالت امن واقع است، اگر یک **دنباله‌ی امن**[1] وجود داشته باشد. با حالت تخصیص فعلی، دنباله‌ای از نخ‌های $<T_1, T_2, \ldots, T_n>$ یک دنباله‌ی امن محسوب می‌شوند که بتوان منابع درخواستی هر یک از نخ‌های T_i را با منابع در دسترس فعلی و منابع در اختیار تمامی T_jها، با $j < i$ ارضا کرد. در این شرایط، اگر منابع مورد نیاز نخ T_i بلافاصله در دسترس نباشند، در این صورت T_i می‌تواند منتظر شود تا این که تمامی T_jها خاتمه یابند. با خاتمه‌یافتن آنها، نخ T_i می‌تواند منابع موردنیاز خود را به‌دست آورده و با تکمیل کار خود، منابع در اختیار خود را برگردانده و خاتمه بپذیرد. لحظه‌ای که نخ T_i خاتمه یافت، نخ T_{i+1} می‌تواند منابع مورد نیاز خود را به‌دست آورد و الی آخر. اگر چنین دنباله‌ای وجود نداشته باشد، در این صورت حالت سیستم، حالت *ناامن*[2] است.

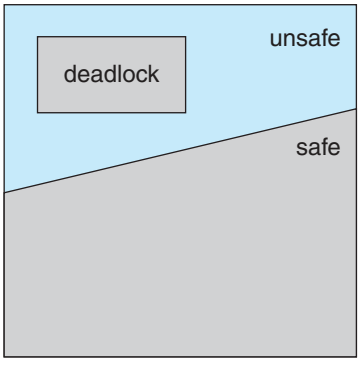
هر حالت ناامن، یک حالت بن‌بست نیست. برعکس، هر حالت بن‌بست، یک حالت ناامن است. با این وجود، تمامی حالت‌های ناامن بن‌بست محسوب نمی‌شوند (شکل ۸-۸). یک حالت ناامن می‌تواند به یک بن‌بست منجر شود. مادامی که حالت امن است، سیستم‌عامل می‌تواند از حالت‌های ناامن (و بن‌بست) اجتناب بورزد. سیستم‌عامل در یک حالت ناامن نمی‌تواند به این خاطر که بن‌بست پیش می‌آید با درخواست منابع نخ‌ها مقابله کند. این رفتار نخ‌هاست که حالت‌های ناامن را کنترل می‌کند.

1. Safe sequence
2. Unsafe

برای روشن‌شدن مطلب، سیستمی را با ۱۲ منبع و سه نخ T_0، T_1 و T_2 در نظر می‌گیریم. نخ T_0 حداکثر به تعداد ده، T_1، چهار و T_2، نه منبع نیاز خواهند داشت. فرض کنید در زمان t_0، نخ T_0 به تعداد پنج ، T_1، دو و T_2، دو منبع در اختیار داشته باشند (بنابراین، سه منبع آزاد است).

	حداکثر نیازها	نیازهای فعلی
T_0	۱۰	۵
T_1	۴	۲
T_2	۹	۲

در لحظه‌ی t_0، سیستم در حالت امن قرار دارد. دنباله‌ی $<T_1, T_0, T_2>$ شرط ایمنی را ارضا می‌نماید. نخ T_1 می‌تواند بلافاصله تمامی منابع خود را در اختیار گرفته و سپس آنها را برگرداند (در اینجا، موجودی سیستم، ۵ منبع خواهد بود)؛ سپس نخ T_0 می‌تواند تمامی منابع لازم خود را گرفته و آنها را پس بدهد (در اینجا، موجودی سیستم، ۱۰ منبع است)؛ و سرانجام نخ T_2 خواهد توانست همگی منابع خود را به‌دست آورده و آنها را بازگرداند (با این کار، سیستم تمامی دوازده منبع خود را موجود می‌بیند).



شکل ۸–۸ فضاهای حالت امن، ناامن و بن‌بست.

یک سیستم می‌تواند از یک حالت امن به یک حالت ناامن برود. فرض کنید نخ T_2 در لحظه‌ی t_1، یک منبع دیگری درخواست نموده و در اختیار می‌گیرد. در اینجا، حالت سیستم دیگر امن نیست. چون تنها می‌توان تمامی منابع نخ T_1 را تخصیص داد. لحظه‌ای که نخ آنها را بر می‌گرداند، موجودی سیستم، تنها ۴ منبع خواهد بود. چون نخ T_0، ۵ منبع در اختیار دارد اما حداکثر نیاز آن ۱۰ می‌باشد، این امکان وجود دارد که هر ۵ منبع مورد نیاز خود را درخواست نماید. چون این تعداد منبع موجود نیست، نخ T_0 به‌ناچار منتظر می‌ماند. به طور مشابه، نخ T_2 نیز ممکن است هر ۶ منبع مورد نیاز خود را درخواست نموده و ناگزیر منتظر شود که پیامد آن بن‌بست خواهد بود. اشتباه ما این بود که درخواست نخ T_2 را برای منبع بیشتر اعطا کردیم. اگر T_2 را منتظر نگه می‌داشتیم تا این که یکی از نخ‌های دیگر خاتمه می‌یافت و منابع خود را رها می‌کرد، در این صورت می‌توانستیم از بن‌بست به دور باشیم.

با داشتن مفهوم حالت امن می‌توانیم الگوریتم‌های اجتناب از بن‌بست را تعریف کنیم که تضمین می‌کنند سیستم هرگز با بن‌بست مواجه نخواهد شد. ایده آن است که به سادگی تضمین شود سیستم همواره در یک حالت امن باقی خواهد ماند. سیستم در ابتدا در یک حالت امن واقع است. هر بار که یک نخ، منبع موجودی را درخواست می‌نماید، سیستم بایستی تصمیم بگیرد که آیا می‌توان منبع را بلافاصله تخصیص داد یا بایستی نخ منتظر شود. تنها در صورتی درخواست اعطا می‌شود که تخصیص آن، سیستم را در یک حالت امن باقی بگذارد.

در این طرح، اگر یک نخ، منبعی را درخواست نماید که فعلاً نیز موجود است، باز هم ممکن است منتظر شود. بنابراین، بهره‌وری منبع در مقایسه با نداشتن این طرح ممکن است پایین‌تر بیاید.

۸-۶-۲ الگوریتم گراف تخصیص منبع

اگر یک سیستم تخصیص منبع داشته باشیم که در آن تنها یک نمونه از هر نوع منبع وجود دارد، برای اجتناب از بن‌بست می‌توانیم گونه‌ای از گراف تخصیص منبع را که در بخش ۸-۳-۲ تعریف شد، استفاده کنیم. علاوه بر یال‌های درخواست و تخصیص که پیشتر بیان کردیم، یک نوع یال جدید به نام **یال ادعا**[1] معرفی می‌کنیم. یال ادعای $T_i \rightarrow R_j$ بیانگر آن است که نخ T_i ممکن است منبع R_j را در زمان آینده درخواست نماید. این یال، هم‌سو با یال درخواست بوده، ولی در گراف با یک بردار خط‌چین نمایش داده می‌شود. لحظه‌ای که T_i منبع R_j را درخواست می‌کند، یال ادعای $T_i \rightarrow R_j$ به یال درخواست نظیر تبدیل می‌شود. به طور مشابه، زمانی که منبع R_j توسط نخ T_i رها می‌گردد، یال تخصیص $R_j \rightarrow T_i$ باز هم به یال ادعای $T_i \rightarrow R_j$ تبدیل می‌شود.

توجه داشته باشید که منابع سیستم بایستی پیشاپیش ادعا شوند. یعنی، پیش از این که T_i اجرای خود را شروع کند، تمامی یال‌های ادعای آن در گراف تخصیص منبع ظاهر شوند. می‌توانیم این شرط را خفیف‌تر کنیم و تنها در صورتی اجازه دهیم تا یال ادعای $T_i \rightarrow R_j$ به گراف افزوده شود که تمامی یال‌های پیشین مربوط به نخ T_i یال‌های ادعا باشند.

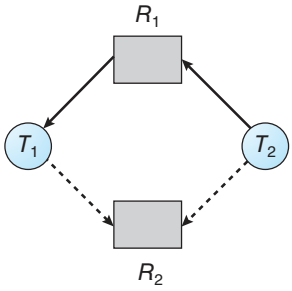
حال فرض کنید نخ T_i منبع R_j را درخواست می‌نماید. تنها در صورتی درخواست قابل اعطا می‌باشد که تبدیل یال درخواست $T_i \rightarrow R_j$ به یال تخصیص $R_j \rightarrow T_i$ باعث پیدایش یک دور در گراف تخصیص منبع نگردد. توجه داشته باشید که امن‌بودن سیستم را با استفاده از یک الگوریتم تشخیص دور بررسی می‌کنیم. الگوریتمی که وجود یک دور را در این گراف تشخیص می‌دهد از پیچیدگی عملیات مرتبه‌ی n^2 برخوردار است که در آن، n تعداد نخ‌های سیستم است.

اگر هیچ دوری وجود نداشت، در این صورت تخصیص منبع، سیستم را در یک حالت امن باقی می‌گذارد. اگر دوری به وجود بیاید، در این صورت تخصیص منبع، سیستم را به یک حالت ناامن خواهد برد. لذا، نخ T_i به ناچار بایستی برای ارضای درخواست‌های خود منتظر شود.

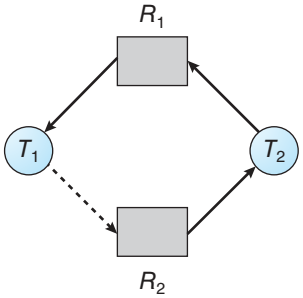
برای روشن‌شدن این الگوریتم، گراف تخصیص منبع شکل ۸-۹ را در نظر می‌گیریم. فرض کنید، T_2 منبع R_2 را درخواست می‌کند. به رغم این که R_2 فعلاً آزاد است، نمی‌توانیم آن را به T_2 تخصیص بدهیم، چرا که با این عمل، یک دور در گراف به وجود می‌آید (شکل ۸-۱۰). همان‌گونه

1. Claim edge

که متذکر شدیم، وجود یک دور بیانگر آن است که سیستم در یک حالت ناامن قرار دارد. اگر T_1 منبع R_2 و T_2 منبع R_1 را درخواست کنند، در این صورت یک بن‌بست اتفاق خواهد افتاد.



شکل ۸-۹ گراف تخصیص منبع برای اجتناب از بن‌بست.



شکل ۸-۱۰ یک حالت ناامن در یک گراف تخصیص منبع.

۸-۶-۳ الگوریتم بانکدار

الگوریتم گراف تخصیص منبع به یک سیستم تخصیص منبع با تعدد نمونه‌های نوع منابع قابل اعمال نیست. الگوریتم اجتناب از بن‌بستی که در ادامه مطرح می‌کنیم برای چنین سیستمی قابل اعمال است، اما کارآمدی آن از طرح گراف تخصیص منبع کمتر است. این الگوریتم عموماً به **الگوریتم بانکدار**[1] معروف است. دلیل این نام‌گذاری آن است که در یک سیستم بانکی با اعمال این الگوریتم می‌توان مطمئن بود که یک بانک هرگز کل موجودی خود را آن‌گونه اعطا نمی‌کند که نتواند نیاز تمامی مشتریان خود را ارضا نماید.

هرگاه نخ جدیدی به سیستم وارد می‌شود، بایستی حداکثر نیاز خود را از نمونه‌های انواع منابع که ممکن است در آینده استفاده کند، اعلام نماید. این تعداد نمی‌تواند از کل تعداد منابع موجود سیستم فراتر رود. زمانی که کاربری مجموعه‌ای از منابع را درخواست می‌نماید، سیستم می‌بایست تعیین کند که آیا تخصیص این منابع، سیستم را در یک حالت امن باقی می‌گذارد. اگر

1. Banker's algorithm

این گونه باشد، منابع تخصیص می‌یابند؛ در غیر این صورت، نخ بایستی منتظر شود تا این که منابع کافی از نخ دیگری رها شوند.

برای پیاده‌سازی الگوریتم بانکدار لازم است که ساختمان‌داده‌های متعددی نگهداری شوند. این ساختمان‌داده‌ها حالت سیستم تخصیص منبع را تعریف می‌کنند. به ساختمان‌داده‌های زیر نیاز داریم که در آن، n، تعداد نخ‌های سیستم و m، تعداد انواع منابع می‌باشد:

- **Available**: برداری به طول m که تعداد منابع موجود هر یک از انواع منابع را نشان می‌دهد. اگر Available[j] برابر k باشد، در این صورت k نمونه از منبع نوع R_j موجود است.
- **Max**: ماتریس $n \times m$ که حداکثر نیاز هر یک از نخ‌ها را تعریف می‌کند. اگر Max[i][j] برابر k باشد، در این صورت نخ T_i می‌تواند حداکثر k نمونه از منبع نوع R_j را درخواست کند.
- **Allocation**: ماتریس $n \times m$ که تعداد تخصیصی فعلی هر یک از انواع منابع برای نخ‌ها را تعریف می‌کند. اگر Allocation[i][j] برابر k باشد، در این صورت فعلاً k نمونه از منبع نوع R_j در اختیار نخ T_i قرار دارد.
- **Need**: ماتریس $n \times m$ که نیاز باقی هر یک از نخ‌ها را از منابع نشان می‌دهد. اگر Need[i][j] برابر k باشد، در این صورت نخ T_i می‌تواند k نمونه‌ی دیگر از منبع نوع R_j را برای تکمیل کار خود درخواست نماید. توجه داشته باشید که Need[i][j] برابر Max[i][j] - Allocation[i][j] است.

اندازه و مقادیر هر یک از این ساختمان‌داده‌ها با گذشت زمان توأمان تغییر می‌کنند.

در ادامه، برای نمایش ساده‌تر الگوریتم بانکدار از یک سری نماد استفاده می‌کنیم. اگر X و Y بردارهایی به طول n باشند. می‌گوییم $X \leq Y$ است، اگر و تنها اگر به ازای تمامی iها، $i = 1, 2, \ldots, n$، $X[i] \leq Y[i]$ باشد. برای مثال، اگر $X = (1,7,3,2)$ و $Y = (0,3,2,1)$ باشد، در این صورت $Y \leq X$ است. به علاوه، اگر $Y \leq X$ و $Y \neq X$ باشد آنگاه، $Y < X$.

می‌توانیم هر یک از سطرهای ماتریس‌های Allocation و Need را بردار فرض کنیم و آنها را به صورت $Allocation_i$ و $Need_i$ به کار ببریم. بردار $Allocation_i$ منابع فعلی در اختیار نخ T_i را مشخص می‌کند؛ بردار $Need_i$ منابع دیگری را که نخ T_i می‌تواند برای تکمیل کار خود درخواست نماید، مشخص می‌سازد.

۸-۶-۳-۱ الگوریتم ایمنی

اکنون الگوریتمی ارائه می‌دهیم که با استفاده از آن می‌توان دریافت که آیا یک سیستم در حالت امن قرار دارد یا خیر. این الگوریتم به صورت زیر بیان می‌شود:

۱- بردارهای Work و Finish را به ترتیب با طول‌های m و n در نظر بگیرید. $Work \leftarrow Available$ و برای $i = 0$ تا $i = n-1$، $Finish[i] \leftarrow false$.

۲- یک اندیس i پیدا کنید که دو شرط زیر را با هم داشته باشد:

a. $Finish[i] == false$

b. $Need_i \le Work$

اگر یک i با این شرایط نبود، برو به مرحله‌ی ۴.

۳- $Work \leftarrow Work + Allocation_i$، $Finish[i] \leftarrow true$ و برو به مرحله‌ی ۲.

۴- اگر برای تمامی iها، $Finish[i] == true$ بود، در این صورت سیستم در یک حالت امن قرار دارد.

این الگوریتم برای تعیین امن‌بودن حالت سیستم به پیچیدگی عملیاتی از مرتبه‌ی $m \times n^2$ نیاز دارد.

۸-۶-۳-۲ الگوریتم درخواست منبع

در ادامه، الگوریتمی برای تعیین این که آیا درخواست‌ها می‌توانند به‌صورت ایمن اعطا شوند یا خیر، مطرح می‌کنیم. فرض کنید $Request_i$، بردار درخواست نخ T_i باشد. اگر $Request_i [j] == k$ باشد، در این صورت نخ T_i، k نمونه‌ی دیگر از منبع نوع R_j را درخواست نموده است. زمانی که درخواستی از جانب نخ T_i انجام می‌شود، اَعمال زیر صورت می‌گیرند:

۱- اگر $Request_i \le Need_i$ بود، آنگاه برو به مرحله‌ی ۲. در غیر این صورت، یک وضعیت خطا گزارش بده، چرا که نخ از حداکثر ادعای خود فراتر رفته است.

۲- اگر $Request_i \le Available$ بود، آنگاه برو به مرحله‌ی ۳. در غیر این صورت، T_i بایستی منتظر شود، چرا که منابع درخواستی آن در دسترس نیستند.

۳- با تغییر حالت به صورت زیر، چنان وانمود کنید که منابع درخواستی نخ T_i تخصیص می‌یابند:

$$Available \leftarrow Available - Request_i;$$
$$Allocation_i \leftarrow Allocation_i + Request_i;$$
$$Need_i \leftarrow Need_i - Request_i;$$

اگر حالت تخصیص منبع حاصل امن بود، تراکنش کامل شده و منابع نخ T_i تخصیص می‌یابند. اما، اگر حالت جدید ناامن بود، در این صورت بایستی T_i برای $Request_i$ منتظر مانده و حالت تخصیص منبع قبلی بازیابی شود.

۸-۶-۳-۳ یک مثال روشنگر

برای روشن‌شدن کاربرد الگوریتم بانکدار، سیستمی را با پنج نخ T_0 تا T_4 و سه نوع منبع A، B و C در نظر بگیرید. منبع نوع A دارای ۱۰، B، ۵ و C، ۷ نمونه می‌باشد. فرض کنید که تصویر لحظه‌ای زیر، وضعیت فعلی سیستم باشد:

	Allocation			*Max*			*Available*		
	A	B	C	A	B	C	A	B	C
T_0	۰	۱	۰	۷	۵	۳	۳	۳	۲
T_1	۲	۰	۰	۳	۲	۲			
T_2	۳	۰	۲	۹	۰	۲			
T_3	۲	۱	۱	۲	۲	۲			

T_4	۰	۰	۲	۴	۳	۳

محتوای ماتریس Need به صورت Max – Allocation تعریف شده و به صورت زیر به‌دست می‌آید:

	Need		
	A	B	C
T_0	۷	۴	۳
T_1	۱	۲	۲
T_2	۶	۰	۰
T_3	۰	۱	۱
T_4	۴	۳	۱

ادعا می‌کنیم که حالت فعلی سیستم امن است. در واقع، دنباله‌ی $<T_1, T_3, T_4, T_2, T_0>$ معیارهای ایمنی را ارضا می‌کند. اکنون فرض کنید نخ T_1، نمونه‌ی دیگری از منبع نوع A و دو نمونه از منبع نوع C را درخواست می‌نماید، لذا، $Request_1 = (1,0,2)$ می‌باشد. جهت تصمیم‌گیری درباره‌ی اعطای آنی منابع درخواستی نخ، نخست بررسی می‌کنیم که آیا $Request_1 \leq Available$ می‌باشد، یعنی آیا $(1,0,2) \leq (3,3,2)$، که درست است. سپس وانمود می‌کنیم که درخواست انجام شده و در ادامه، به حالت جدید زیر می‌رسیم:

	Allocation			*Need*			*Available*		
	A	B	C	A	B	C	A	B	C
T_0	۰	۱	۰	۷	۴	۳	۲	۳	۰
T_1	۳	۰	۲	۰	۲	۰			
T_2	۳	۰	۲	۶	۰	۰			
T_3	۲	۱	۱	۰	۱	۱			
T_4	۰	۰	۲	۴	۳	۱			

بایستی تعیین کنیم که آیا حالت جدید سیستم امن می‌باشد یا خیر. برای انجام این کار، الگوریتم ایمنی خودمان را اجرا کرده و به دنباله‌ی $<T_1, T_3, T_4, T_0, T_2>$ می‌رسیم که نیاز ایمنی را ارضا می‌نماید. از این رو، بلافاصله می‌توانیم درخواست نخ T_1 را اعطا کنیم.

با این وجود، بایستی بدانید، زمانی‌که سیستم در این حالت قرار دارد، درخواستی به صورت $(3,3,0)$ از جانب T_4 قابل اعطا نیست، چون منابع کافی موجود نیست. به علاوه، حتی به رغم وجود منبع کافی، درخواستی به صورت $(0,2,0)$ از جانب T_0 هم قابل اعطا نیست، چون به یک حالت ناامن می‌رسیم.

پیاده‌سازی الگوریتم بانکدار را به عنوان یک تمرین برنامه‌نویسی برای دانش‌جویان واگذار می‌کنیم.

۸-۷ تشخیص بن‌بست

اگر سیستمی هیچ یک از روش‌های پیشگیری و یا الگوریتم اجتناب از بن‌بست را به کار نبرد، در